EXPERIMENT 14

PRO-CODE EXTENSION USING CATALYST / CUSTOM FUNCTIONS

Student Name: Dakshesh N **Roll Number:** 241401017

1. Aim

To understand how a Zoho Creator application can be extended with pro-code logic through a custom function or serverless endpoint.

2. Objectives

• Distinguish low-code configuration from pro-code extension.
• Design a secure request/response flow between Creator and an external function.
• Review input validation and JSON response handling.
• Identify what must be configured and tested before a real integration.

3. Platform and Application Context

Platform: Zoho Creator
Application: Loan Application & Pre-Screening Management
Form: Loan_Application_Form
Report: Loan_Application_Report
Student Roll No.: 241401017

4. Theory

Pro-code extensions are useful when application logic requires libraries, runtime control, or specialized processing beyond native low-code features. A typical pattern is Creator → authenticated API request → server-side function → JSON response. Credentials must be stored in secure connection/secret facilities rather than embedded in client-side code. Any financial or eligibility logic in this experiment is illustrative only and must not be used to make real lending decisions.

5. Procedure

1. Design a small demonstration endpoint that accepts monthlyIncome and requestedAmount.
2. Validate that both inputs are numeric and positive; reject invalid inputs with HTTP 400.
3. Calculate a clearly labeled demonstration flag using a simple threshold, and return JSON.
4. Plan how Creator would call the endpoint using an authorized connection or secure API mechanism.
5. Test valid and invalid inputs in a development environment before any integration; record actual status codes and responses if executed.
6. Do not connect to real applicant data or make a real credit decision with the demonstration logic.

6. Observations / Test Results

Area	Observation	Status / limitation
Architecture	Creator sends selected fields to a server-side function.	Illustrative architecture only.
Input validation	Sample code checks numeric, positive values.	Code example supplied; not run against a live endpoint.
Authentication	Use platform-managed connection/secret configuration.	No live credentials or endpoint configured.
Execution	No Catalyst project/function deployment was provided.	Live invocation not tested.

7. Result

The experiment presents a pro-code extension pattern and an instructional JavaScript example for input validation and JSON output. It does not claim that a Catalyst project or live API integration was deployed.

8. Conclusion

Hybrid development allows Zoho Creator to manage forms and workflows while a server-side function handles specialized logic. Secure authentication, validation, error handling, and real tests are required before production use.

Illustrative JavaScript code (not deployed)

```
// Illustrative server-side JavaScript function (not deployed)
// Input: { "monthlyIncome": 60000, "requestedAmount": 250000 }
// Demo-only rule; not a real credit decision.

function handler(request, response) {
  const body = request.body || {};
```

```
  const income = Number(body.monthlyIncome);
  const amount = Number(body.requestedAmount);

  if (!Number.isFinite(income) || !Number.isFinite(amount) ||
      income <= 0 || amount <= 0) {
    response.setStatus(400);
    response.setHeader("Content-Type", "application/json");
    response.write(JSON.stringify({ error: "Invalid input" }));
    return;
  }

  const demoFlag = amount <= income * 6 ? "WITHIN_DEMO_LIMIT" : "REVIEW";
  response.setHeader("Content-Type", "application/json");
  response.write(JSON.stringify({
    status: "OK",
    demoFlag: demoFlag,
    note: "Demonstration only; not a lending recommendation."
  }));
}
```

Expected conceptual tests

Input	Expected response	Status
monthlyIncome=60000, requestedAmount=250000	JSON with status OK and demoFlag WITHIN_DEMO_LIMIT	Expected only; not executed
monthlyIncome=-1, requestedAmount=250000	JSON error: Invalid input; HTTP 400	Expected only; not executed

9. Evidence

The figure below is a generated explanatory schematic based on the documented experiment context. It is clearly labeled illustrative and is not a captured screenshot of a live deployment or a claim that untested actions succeeded.

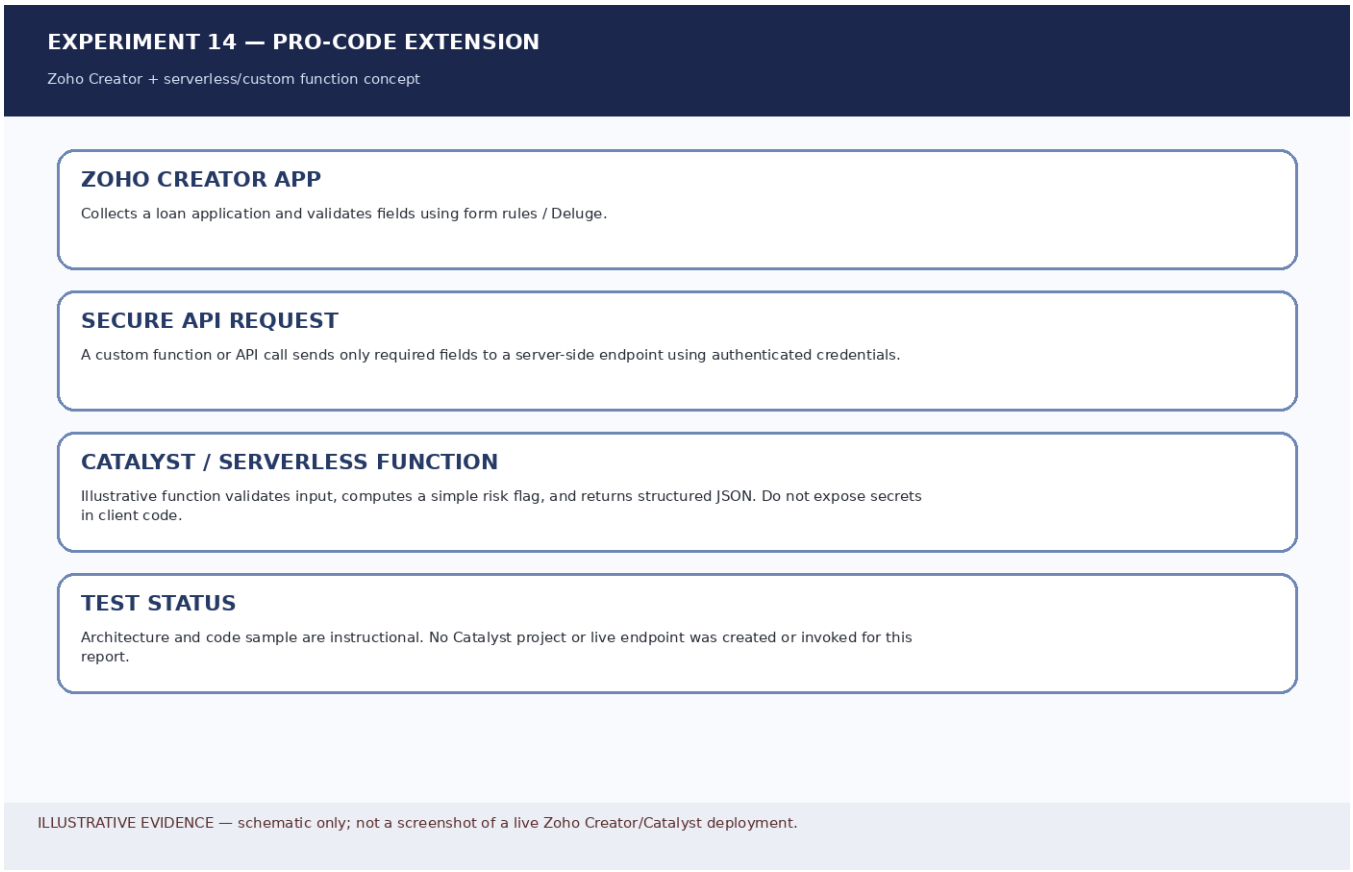


Figure 14.1 — Illustrative Creator-to-serverless architecture; not a live integration.

Submission note: Where the experiment requires live configuration evidence, replace or supplement this schematic with your own screenshot from Zoho Creator/Catalyst after completing the corresponding step. Do not submit the schematic as a real system screenshot.